

Security Engineering

Flask Tutorial 3

Hoang Nguyen Srđan Krstić Daniel Galán
Institute of Information Security
Department of Computer Science
ETH Zürich

Before starting this assignment, make sure you have completed the two Flask tutorials available on the Moodle page. Reminder: You can get help for this assignment in the lab sessions every *Friday, 10:15-12:00, in CHN D 46*.

In the previous Flask tutorials, you have implemented authorization policies manually in the Thoughts web application. Such security policies can also be implemented more systematically, e.g., by using existing libraries.

In this lab assignment, you will first implement security requirements (authorization in particular) and then privacy requirements (data protection in particular) in the Thoughts application. For the former, you will familiarize yourself with several libraries for writing and enforcing authorization policies. For the latter, you will manually implement measures for handling personal data in compliance with the GDPR. There, we will focus on the principles of purpose limitation and data subject consent.

We will start from the solution of the last tutorial, which we will refer to as `Thoughts-sec-base`.

Contents

1	Security Requirements	2
1.1	Introduction	2
1.2	Role-based Access Control (RBAC)	3
1.2.1	Implementing RBAC Manually	3
1.2.2	Flask-User: Role-based access control	4
1.3	Fine-grained Access Control (FGAC)	7
1.4	Security policies for confidentiality	7

2	Privacy requirements	8
2.1	Privacy policy declaration	9
2.2	Defining purposes	9
2.3	Identify personal data	9
2.4	Consent management	10

1 Security Requirements

1.1 Introduction

In this assignment, we will continue working with the Thoughts application as our running example. Thoughts is a simple web application that allows users to create thoughts, share them with others, and vote for their favorites.

In this (updated) version of the application, authenticated users can have either the `USER` or `OTHER` role. We also extend the authorization policies as:

- (SP1) Any user can read the home page
- (SP2) Authenticated users **with the USER role** can create thoughts.
- (SP3) Authenticated users **with the USER role** can delete **their own** thoughts.
- (SP4) Authenticated users **with the USER role** can vote **at most three times for the same thought**.
- (SP5) Authenticated users **with the USER role** are not allowed to vote for **their own** thoughts.

The defined policies must be enforced on specific routes within the application. (SP1) applies to the `/index` route. (SP2) applies the `/add_thought` route. (SP3) applies the `/delete_thought` route. Finally, (SP4) and (SP5) relates to the `/vote_thought` route.

Each policy may involve three components: authentication requirements, **role-based access control**, and **fine-grained authorization constraints**.

In the previous tutorial, we learned how to enforce authentication either manually or using the Flask-User library. We also implemented fine-grained authorization checks manually, invoked them at the appropriate points.

In this tutorial, we will learn how to implement the role-based access control policy manually and also using a library. Next, we will learn how to implement fine-grained access control policy in a more systematic way.

Finally, we introduce further confidentiality constraints and demonstrate how to restrict unauthorized read access.

1.2 Role-based Access Control (RBAC)

Role-based access control (RBAC) is an access control model that restricts system access based on a user's role. Instead of assigning permissions to users directly, RBAC groups users into roles, typically aligned with their business functions, and assigns a predefined access rights to each role.

Our starting point is last week's final task implementation of the Thoughts app. In this section, we first show initial code changes to this iteration and then use it as a base for the upcoming tasks.

You can either use your own implementation and apply the described changes manually, or download `thoughts-sec-template` from Moodle, which includes our solution with all changes already incorporated. To avoid having to copy code from this PDF, you can copy the patches from the provided template instead. Tutorial explanations are based on our provided template.

1.2.1 Implementing RBAC Manually

On top of last week's iteration we make the following changes:

```
+ from enum import Enum, auto
...
+ class Role(Enum):
+     USER = auto()
+     OTHER = auto()
...
class Person(db.Model, UserMixin):
+     myRole = db.Column(db.Enum(Role), default=Role.USER)
...
```

Namely, the base implementation introduces a new `Role` enumeration to define possible user roles, `USER` and `OTHER`. In the `Person` class, a new column `myRole` is added to store each user's role. By default, new users are assigned the `USER` role.

Since (SP1) remains unchanged, we consider now the (SP2) policy. While the `@login_required` decorator ensures that an authenticated user accesses the route, it does not verify the user's role. To enforce (SP2), we manually implement a role check to ensure that only users with the `USER` role can create new thoughts:

```
class Person(db.Model, UserMixin):
    ...
+     def isUser(self):
+         return self.myRole == Role.USER
    ...
+     def check_user(caller):
```

```

+ if not caller.isUser():
+     raise SecurityException("You are not allowed to access this page: Not a
      User")
...
@app.route('/add_thought', methods=['POST'])
@login_required
def add_thought():
+ try:
+     check_user(current_user)
      content = request.form["thought"]
      ...
+ except SecurityException as se:
+     thoughts = Thought.query.all()
+     return render_template('index.html', table=thoughts,
+         security_violation=True, message=se.msg)

```

We define an explicit authorization check, `check_user`, which should be used in endpoints accessible only to logged in users (as shown for the `/add_thought` endpoint above). This check verifies that the logged in user has the role `USER`, as opposed to the `OTHER` role. If a user without the required role attempts to access this endpoint, the new thought will not be added, and the user will be redirected to the index page with an error message shown.

Task 1: Implement the RBAC checks manually

Retrieve last week's task implementation to build upon it. Use your own work or download the provided `thoughts-sec-template` from Moodle. If using your own work, integrate the code changes above into your code before starting this task.

.....
Implement the remaining policies using RBAC checks as described.

.....
After the corresponding deadline, the solution will be published in Moodle as `thoughts-sec-manual`.

Next, we review security features of several existing libraries that support enforcing authorization policies, role-based and fine-grained in Flask applications, and if feasible, demonstrate their usage on one of these policies.

1.2.2 Flask-User: Role-based access control

In the last Flask tutorial, we have introduced the Flask-User library for user authentication and credential management features. In addition to that, Flask-User also provides other security features, including role-based access

control (RBAC). Similarly, the following libraries also provide support for the same class of authorization: Flask-RBAC ¹, Flask-Authorize ².

Installation Flask-User is pre-installed in the Docker template, or available via `requirements.txt` for local setups.

Example We now explain RBAC using Flask-User with an example. You can find this example with all code changes already applied in Moodle, under the name `thoughts-sec-flaskuser`.

Since some of the aforementioned policies (i.e. (SP2)-(SP5)) apply to the users with `USER` role, we will enforce these requirements using Flask-User.

First, let us demonstrate how to create a `Role` class and associate it with the current user class. We start this step by modifying the current code snippet in the application:

```
...
- class Role(Enum):
-     USER = auto()
-     OTHER = auto()
+ USER = "User"
+ OTHER = "Other"

class Person(db.Model, UserMixin):
    id = db.Column(db.Integer, primary_key=True)
    ...
- myRole = db.Column(db.Enum(Role), default=Role.USER)
+ roles = db.relationship('Role', secondary='user_roles')

+ class Role(db.Model):
+     id = db.Column(db.Integer, primary_key=True)
+     name = db.Column(db.String(50), unique=True)

+ class UserRoles(db.Model):
+     id = db.Column(db.Integer, primary_key=True)
+     user_id = db.Column(db.Integer, db.ForeignKey('person.id'), unique=True)
+     role_id = db.Column(db.Integer, db.ForeignKey('roles.id'))
...

with app.app_context():
    db.create_all()
+     roles = Role.query.all()
+     if len(roles) == 0:
+         db.session.add(Role(name=USER))
+         db.session.add(Role(name=OTHER))
+         db.session.commit()
```

¹Flask-RBAC documentation: <https://flask-rbac.readthedocs.io/en/latest/>.

²Flask-Authorize documentation: <https://flask-authorize.readthedocs.io/en/latest/>.

In this case, instead of defining an enum class, we define a proper `Role` class: A role contains an id and a name. By convention of Flask-User library, we create also a `UserRoles` association between `User` and `Role` classes, and we add the constraint that a user can only have one role (i.e. making the association-end unique). Note that with this approach, one also has to initialize the `Role` objects when the database is first created.

Next, in order to enforce the role-based authorization policy, we include the `@roles_required` annotation over target routes, assign our default error endpoint for handling unauthorized access, and assign appropriate User role when a new user is registered. In our example, we remove the manual `check_user` check with the `@roles_required(USER)` annotation.

```
- from flask_user import login_required, UserManager, UserMixin, current_user
+ from flask_user import login_required, UserManager, UserMixin, current_user, \
+     roles_required, user_registered
...
+ app.config['USER_UNAUTHORIZED_ENDPOINT'] = 'error'
...
+ @user_registered.connect_via(app)
+ def _after_user_registered_hook(sender, user, **extra):
+     role = Role.query.filter_by(name=USER).one()
+     user.roles.append(role)
+     db.session.commit()
...
+ @app.route('/error')
+ def error():
+     msg = "You are not allowed to access this page: Not a User"
+     return render_template('error.html', message = msg)
...
@app.route('/add_thought', methods=['POST'])
@login_required
+ @roles_required(USER)
def add_thought():
-     try:
-         check_user(current_user)
-         content = request.form["thought"]
-         ...
-     except SecurityException as se:
-         return render_template('error.html', message = msg)
+ @roles_required(USER)
def index():
```

This approach is well-suited for applications that require role-based access control. The code becomes simpler and easier to maintain as one does not need to explicitly call the `check_user` function and repeat the exception handling for each route.

However, Flask-User is clearly not expressive enough to enforce the required authorization policies. Therefore, we now turn to the next library that offers more fine-grained access control support.

1.3 Fine-grained Access Control (FGAC)

In this section, we introduce another access control model called **fine-grained access control (FGAC)**. Unlike the RBAC model, where authorization policies depend only on static information, i.e., the assignments of users and permissions to roles, the FGAC policies also depend on dynamic information, such as the satisfaction of authorization constraints in the current context of the system. For example: “*Finance staff may view invoices only under \$10k from a corporate IP address*”. It is common to layer FGAC policies on top of RBAC, i.e., to combine role-based and context-driven policies.

Because the paper focuses on privacy rather than access control, we omit the fine-grained access-control material from the tutorial to keep the artifact concise and focused on the study’s main objectives.

1.4 Security policies for confidentiality

So far, all policies have referred to integrity of the data, i.e., who can create, modify, or delete thoughts and votes. We now consider the confidentiality of the data, i.e., who can read it. Note that, even though it concerns to “reading” of the data, (SP1) is an availability policy. Now consider the following security policy:

(SP6) Any user can read the thought’s content, but only authenticated user with role USER can read the thought’s creator.

To enforce the given policy, we need to modify the `index` route to check the user’s role before passing the list of thoughts to the `render_template` function.

```
@app.route('/')
def index():
    thoughts = Thought.query.all()
    + if not (current_user.is_authenticated and USER in [r.name for r in
current_user.roles]):
    +     for thought in thoughts:
    +         thought.createdBy.username = "RESTRICTED"
    return render_template('index.html', table=thoughts)
```

More specifically, we modify the route so that the creator’s username is only visible to authenticated users with the `USER` role. If a user does not meet this requirement, the creator’s username is replaced with the string “RESTRICTED” before rendering the template. While this approach works for enforcing the given confidentiality constraint, it is not very systematic.

A more structured and reusable solution is to introduce Data Transfer Object (DTO). Whenever an object is retrieved from the database, it is converted into a DTO before being passed to the template. The DTO is responsible for exposing only the appropriate subset of data, with respect to the confidentiality policies.

Task 2: Implementing DTO for confidentiality constraints

Download the template `thoughts-confidentiality-template` from Moodle.

.....
Implement a DTO solution for the Thought object. To achieve this, define a `ThoughtDTO` class that contains the same fields as the original `Thought` class. In the `/index` route, instead of returning raw `Thought` objects, you should convert each to a collection of `ThoughtDTO` objects, applying the appropriate confidentiality checks during the conversion. We provide you with a predefined `RESTRICTED` object, importable from the `dto.py` file, that should be used to mask restricted attributes association-ends. Note that fields that are collections should be initialized as empty lists (`[]`) if they are restricted.

.....
After the corresponding deadline, the solution will be published in Moodle as `thoughts-confidentiality`.

2 Privacy requirements

In accordance with privacy regulations, such as the General Data Protection Regulation (GDPR), the application must enforce data privacy principles that ensure personal data is collected and processed for specific purposes; and with the informed consent of the data subject.

This section presents a concrete approach to implement these requirements within the Thoughts application. Specifically, we extend the base implementation to define privacy-related concepts and functionality. Recall that the base implementation is located in the subdirectory `/phil-base` within the downloaded archive.

2.1 Privacy policy declaration

As presented in lectures and exercises, a privacy policy (or notice) is a list of declared purposes (a tuple of a personal data and a purpose) shown to the user.

Our starting point is `thoughts-sec-manual-template`, which can be downloaded from Moodle. In this section, we interactively add code to this template. To avoid having to copy code from this PDF, we also provide a second template `thoughts-privacy-template` with all changes incorporated.

(PP1) We use the information on the thoughts you have created to add new thoughts.

2.2 Defining purposes

Suppose that the Thoughts application supports the following specific purpose for processing personal data: Adding new thoughts. To model this purpose explicitly in the application, we define a `Purpose` enumeration as follows:

```
+ class Purpose(Enum):  
+     ADD_NEW_THOUGHTS = auto()
```

2.3 Identify personal data

Next, we need to identify which elements of the application represent personal data. For example, their association end, such as the thoughts they have created, can be considered personal data. These elements are modeled in the following enumeration:

```
+ class PersonalData(Enum):  
+     PERSON_CREATED = auto()
```

When a user attempts to perform an action, such as creating a new thought, the application must ensure that, in addition to the existing access control checks, the use of any personal data involved in the action is explicitly declared in the privacy policy, and the data subject has provided consent for that specific usage.

2.4 Consent management

To capture the user's consent, we define a `Consent` class that represents the relationship between the user, the personal data being processed, and the purpose for which it is used. The class is defined as follows:

```
+ class Consent(db.Model):  
+     id = db.Column(db.Integer, primary_key=True)  
+     person_id = db.Column(db.Integer, db.ForeignKey('person.id'))  
+     purpose = db.Column(db.Enum(Purpose), nullable=False)  
+     personalData = db.Column(db.Enum(PersonalData), nullable=False)
```

This class ensures that consent is explicitly recorded for each user, their personal data, and specific purpose. To allow users to dynamically manage their consents, we implement functions to grant, revoke, and verify consents.

```
def grant_consent(person, personal_data, purpose):  
    existing = Consent.query.filter_by(  
        person_id=person.id,  
        personalData=personal_data,  
        purpose=purpose  
    ).first()  
    if not existing:  
        consent = Consent(  
            person_id=person.id,  
            personalData=personal_data,  
            purpose=purpose  
        )  
        db.session.add(consent)  
        db.session.commit()  
  
def revoke_consent(person, personal_data, purpose):  
    consent = Consent.query.filter_by(  
        person_id=person.id,  
        personalData=personal_data,  
        purpose=purpose  
    ).first()  
    if consent:  
        db.session.delete(consent)  
        db.session.commit()  
  
def has_consent(person, personal_data, purpose):  
    return Consent.query.filter_by(  
        person_id=person.id,  
        personalData=personal_data,  
        purpose=purpose  
    ).first() is not None
```

We also provide a dedicated endpoint and the corresponding HTML page that allow users to view the privacy policy and update their consents:

```
@app.route('/policy', methods=['GET', 'POST'])  
@login_required  
def policy():
```

```

if request.method == 'POST':
    create_consent = request.form.get('create_consent') == 'on'
    if create_consent:
        grant_consent(current_user,
            PersonalData.PERSON_CREATED, Purpose.ADD_NEW_THOUGHTS)
    else:
        revoke_consent(current_user,
            PersonalData.PERSON_CREATED, Purpose.ADD_NEW_THOUGHTS)
    return redirect(url_for('policy'))
return render_template('policy.html',
    PersonalData=PersonalData,
    Purpose=Purpose,
    has_consent=has_consent)

```

```

<h2>Privacy policy</h2>

```

```

<form method="POST">
    <label>
        <input type="checkbox" name="create_consent"
            {% if has_consent(current_user,
                PersonalData.PERSON_CREATED, Purpose.ADD_NEW_THOUGHTS) %}
                checked
            {% endif %}>
        I consent to linking my identity as the author to
        thoughts I create, for the purpose of creating new thoughts.
    </label>
    <br>
    <br>
    <input type="submit" value="Update">
</form>

```

Task 4: Implement the consent checks

Download the template `thoughts-privacy-template` from the “Resources” section on Moodle.

.....

Implement the appropriate consent check to make sure that the application enforces the declared privacy policies.

.....

After the corresponding deadline, the solution will be published in Moodle as `thoughts-privacy`.